Funkcjonalność

System oferuje zestaw funkcjonalności niezbędnych do pracy zespołowej nad projektami — od bezpiecznego logowania, przez zarządzanie projektami i zadaniami, aż po narzędzie administracyjne. Poniżej zwięzłe podsumowanie tego, co zostało zaimplementowane:

Bezpieczeństwo i uwierzytelnianie

- Rejestracja oraz logowanie z obsługą JWT, co pozwala na autoryzację żądań.
- Opcjonalne dwuetapowe uwierzytelnianie (2FA) podnoszące poziom ochrony kont.
- Wprowadzenie systemu ról użytkowników (np. USER, MANAGER ,ADMIN), który umożliwia kontrolowanie dostępu do określonych zasobów i operacji w systemie.

Zarządzanie projektami

- Pełny zestaw operacji CRUD (Create-Read-Update-Delete) na projektach.
- Rozszerzona logika kontroli dostępu i ról użytkowników; administrator może precyzyjnie definiować uprawnienia.

Zarządzanie zadaniami

- Endpointy do tworzenia, edycji, usuwania i podglądu zadań w ramach projektów.
- Możliwość przypisania zadań konkretnym użytkownikom, ustawienia terminów oraz śledzenia statusu i priorytetu.
- Filtrowanie zadań według różnych kryteriów (status, projekt, priorytet), co ułatwia szybkie wyszukiwanie.

Panel administratora

- Podgląd wszystkich kont użytkowników z opcją modyfikacji ich ról.
- Rozbudowane reguły autoryzacji – każdy wrażliwy endpoint chroniony jest przed nieuprawnionym dostępem.

Integracja z interfejsem użytkownika

- Wszystkie powyższe operacje są obsługiwane w aplikacji frontendowej React; użytkownik może zarządzać projektami i zadaniami bez odświeżania strony, a zmiany pojawiają się w czasie rzeczywistym.

Jakość i testowanie

- Pokrycie kodu testami jednostkowymi Spring Boot, co zabezpiecza i ułatwia dalszy rozwój.

Struktura projektu

- BACKEND

```
backend/
├── src/
   ├── main/
   │   ├── java/
   │   │   └── pl/projekt/backend/
   │   │       ├── config/
   │   │       ├── controller/
   │   │       ├── dto/
   │   │       ├── exception/
   │   │       ├── model/
   │   │       ├── repository/
   │   │       ├── security/
   │   │       └── service/
   │
   │
   │
   └── test/
      └── java/pl/projekt/backend/
         ├── controller/
         └── service/
```

> **src/main/java/pl/projekt/backend/**

Główna przestrzeń kodu aplikacji:

- config/ – konfiguracje Springa, np. CORS, SecurityConfig.
- controller/ – kontrolery REST API obsługujące żądania HTTP (np. /api/users).
- dto/ – obiekty transferowe (Data Transfer Objects) do komunikacji między frontendem a backendem (np. UserDto, TaskRequest).
- exception/ – obsługa wyjątków i błędów, np. GlobalExceptionHandler, własne klasy błędów.
- model/ – klasy reprezentujące encje bazy danych (np. User, Project, Task).
- repository/ – interfejsy JPA do komunikacji z bazą danych (np. UserRepository, TaskRepository).

- security/ – logika uwierzytelniania/autoryzacji (np. JWT, filtry bezpieczeństwa, klasy UserDetailsService).

- service/ – warstwa logiki biznesowej (np. TaskService, ProjectService, UserService)

> src/test/java/pl/projekt/backend/
Testy jednostkowe:

- controller/ – testy endpointów REST.

- service/ – testy logiki biznesowej

● FRONTEND

```
frontend/
├── node_modules/
├── public/
├── src/
│   ├── api/
│   ├── assets/
│   ├── auth/
│   ├── components/
│   │   └── ui/
│   ├── context/
│   ├── hooks/
│   ├── lib/
│   ├── pages/
│   │   ├── admin/
│   │   ├── auth/
│   │   ├── manager/
│   │   ├── projects/
│   │   └── user/
│   ├── routes/
│   ├── styles/
│   └── utils/
```

> node_modules/
Katalog zainstalowanych zależności npm (generowany automatycznie).

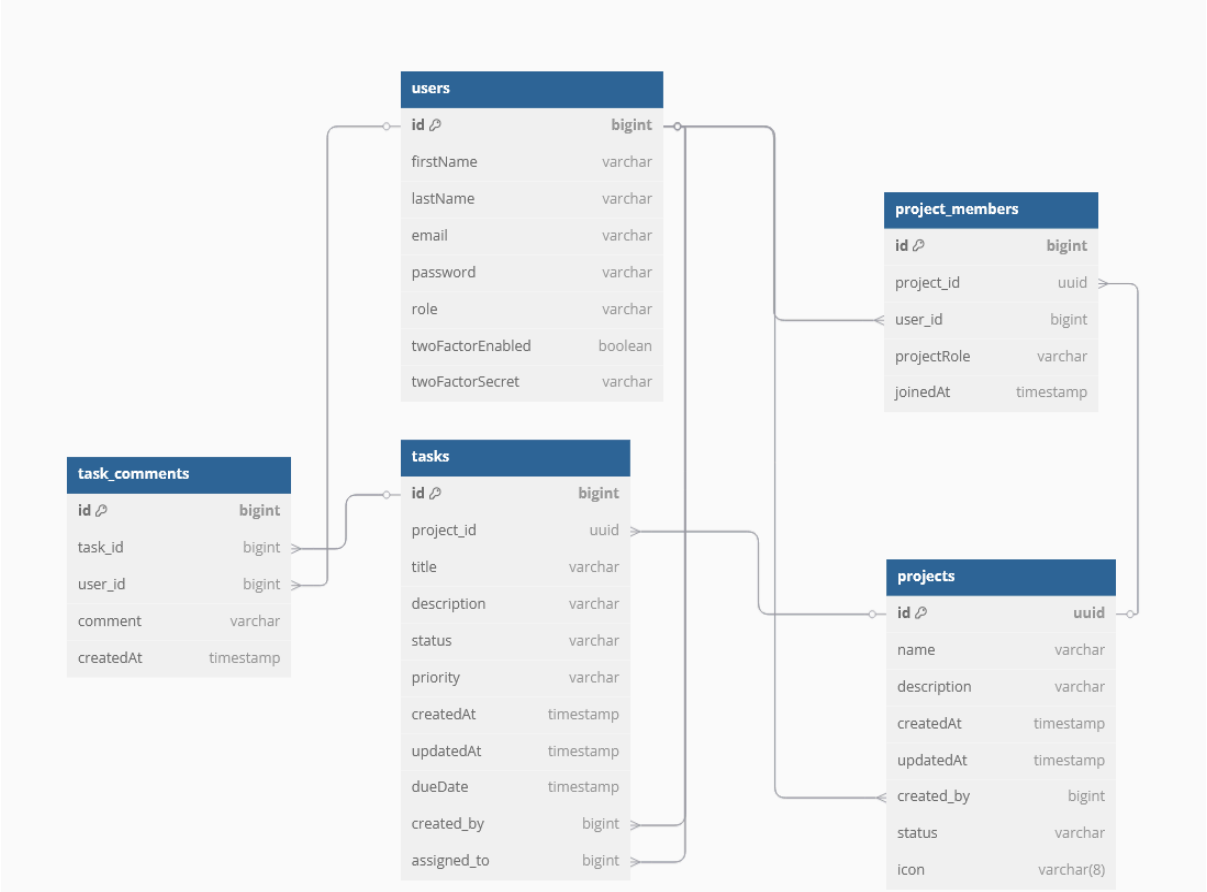
> public/
Pliki publiczne dostępne bezpośrednio (np. index.html, favicon, manifest).

> src/
Kod źródłowy aplikacji React:

- api/ – funkcje do komunikacji z backendem - axios.
- assets/ – pliki graficzne, ikony, czcionki, itp.
- auth/ – logika uwierzytelniania, np. login, tokeny.
- components/ – komponenty UI wykorzystywane w aplikacji.
    - ui/ – komponenty niskiego poziomu (np. przyciski, modale, formularze).
- context/ – konteksty Reacta do zarządzania stanem globalnym.
- hooks/ – niestandardowe hooki React.
- lib/ – biblioteki pomocnicze, np. walidacje, formatowanie dat, klasy pomocnicze.
- pages/ – główne widoki aplikacji, podzielone według ról lub sekcji:
    - admin/ – strony administracyjne (np. zarządzanie użytkownikami).
    - auth/ – logowanie, rejestracja, reset hasła.
    - manager/ – widoki kierownika projektu.
    - projects/ – widoki projektów i zadań.
    - user/ – widoki zwykłego użytkownika.
- routes/ – definicje tras dla React Router.
- styles/ – globalne style CSS lub pliki Tailwind/postCSS.
- utils/ – funkcje pomocnicze.

Baza danych struktura

Baza danych została zaprojektowana na potrzeby aplikacji do zarządzania zadaniami w zespołach projektowych. Głównymi jednostkami są użytkownicy, projekty oraz zadania. Każdy użytkownik posiada podstawowe dane takie jak imię, nazwisko, e-mail, hasło, rola oraz opcjonalnie ustawioną dwuskładnikową autoryzację (2FA). Projekty zawierają informacje takie jak nazwa, opis, status oraz dane o użytkowniku, który je utworzył. Użytkownicy mogą być członkami wielu projektów dzięki tabeli pośredniczącej project_members, która dodatkowo określa rolę użytkownika w projekcie oraz datę dołączenia. W ramach projektów można tworzyć zadania, które zawierają tytuł, opis, priorytet, status, terminy oraz informacje o osobie tworzącej i przypisanej do zadania. Użytkownicy mogą również komentować zadania, a każdy komentarz jest przypisany do konkretnego użytkownika i zadania wraz z datą utworzenia. Relacje między tabelami umożliwiają odwzorowanie struktury zespołów, przypisywanie zadań do konkretnych osób i projektów, a także wspierają współpracę i komunikację w ramach komentarzy. Całość struktury wspiera efektywne zarządzanie projektami i zespołami w aplikacji.



Lista punktów końcowych(API)

/api/auth

- POST /api/auth/register – Rejestracja nowego użytkownika
- POST /api/auth/login – Logowanie użytkownika
- POST /api/auth/2fa/enable – Włączanie dwuskładnikowego uwierzytelniania (2FA)
- POST /api/auth/2fa/verify – Weryfikacja kodu 2FA

/api/users

- GET /api/users/me – Pobieranie podstawowych informacji o aktualnie zalogowanym użytkowniku
- GET /api/users – Pobieranie listy wszystkich użytkowników (tylko MANAGER, ADMIN)

/api/admin

- POST /api/admin/change-role – Zmiana roli użytkownika (tylko ADMIN)
- GET /api/admin/users – Pobieranie wszystkich użytkowników (tylko ADMIN)

/api/projects

- GET /api/projects – Pobieranie wszystkich projektów utworzonych przez aktualnego użytkownika
- GET /api/projects/{id} – Pobieranie projektu po ID
- POST /api/projects – Tworzenie nowego projektu (tylko MANAGER)
- PUT /api/projects/{id} – Aktualizacja projektu (tylko MANAGER)
- DELETE /api/projects/{id} – Usuwanie projektu (tylko MANAGER)

- POST /api/projects/{projectId}/members – Dodawanie użytkownika do projektu (tylko MANAGER)
- GET /api/projects/{projectId}/members – Pobieranie członków projektu
- DELETE /api/projects/{projectId}/members/{userId} – Usuwanie użytkownika z projektu (tylko MANAGER)
- GET /api/projects/my-member-projects – Pobieranie projektów, w których aktualny użytkownik jest członkiem

/api/tasks

- POST /api/tasks – Tworzenie nowego zadania
- PUT /api/tasks/{id} – Aktualizacja zadania
- DELETE /api/tasks/{id} – Usuwanie zadania
- GET /api/tasks/project/{projectId} – Pobieranie zadań dla projektu
- GET /api/tasks/my – Pobieranie zadań zalogowanego użytkownika
- GET /api/tasks/project/{projectId}/all – Pobieranie wszystkich zadań projektu (dla MANAGERA)
- GET /api/tasks/{id} – Pobieranie pojedynczego zadania
- GET /api/tasks/{id}/details – Pobieranie szczegółów zadania z informacją o twórcy
- PATCH /api/tasks/{id}/to-review – Przekazanie zadania do sprawdzenia (zmiana statusu na TO_REVIEW)
- POST /api/tasks/{taskId}/comments – Dodawanie komentarza do zadania
- GET /api/tasks/{taskId}/comments – Pobieranie komentarzy do zadania

Wykonane testy

AdminControllerTest
- Zmiana roli użytkownika na MANAGER
- Błąd przy próbie ustawienia nieprawidłowej roli
- Błąd przy próbie zmiany roli nieistniejącego użytkownika
- Pobieranie listy wszystkich użytkowników

AuthControllerTest
- Rejestracja użytkownika
- Logowanie użytkownika
- Włączanie 2FA
- Weryfikacja 2FA

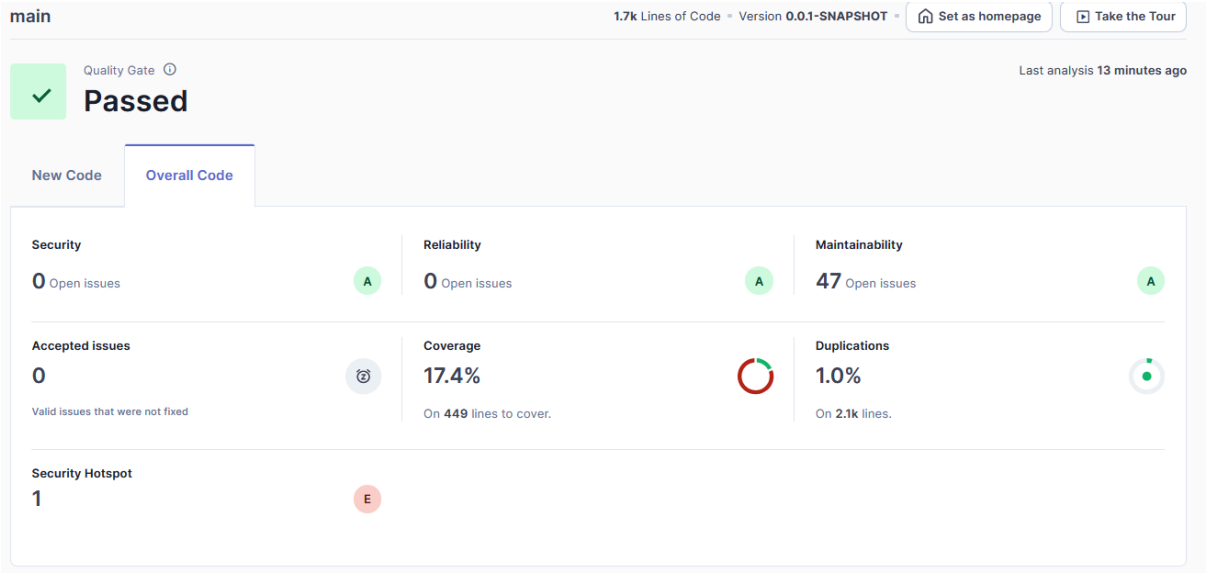
ProjectControllerTest
- Pobieranie wszystkich projektów użytkownika
- Pobieranie projektu po ID
- Pobieranie projektu po ID - brak projektu
- Tworzenie nowego projektu
- Aktualizacja projektu
- Usuwanie projektu
- Dodawanie użytkownika do projektu
- Pobieranie członków projektu
- Usuwanie użytkownika z projektu
- Pobieranie projektów, w których jestem członkiem

TaskControllerTest
- Tworzenie nowego zadania
- Aktualizacja zadania
- Usuwanie zadania
- Pobieranie zadań dla projektu
- Pobieranie zadań zalogowanego użytkownika
- Pobieranie wszystkich zadań projektu (dla MANAGERA)
- Pobieranie pojedynczego zadania
- Pobieranie szczegółów zadania z informacją o twórcy
- Zmiana statusu zadania na TO_REVIEW

UserControllerTest
- Pobieranie informacji o aktualnie zalogowanym użytkowniku
- Pobieranie listy wszystkich użytkowników

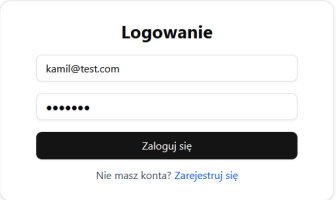
Analiza statyczna kodu

W ramach prac nad projektem przeprowadzono **analizę statyczną kodu** przy użyciu narzędzia **SonarQube**, co pozwoliło na identyfikację potencjalnych błędów, nieczytelnych fragmentów kodu oraz miejsc wymagających refaktoryzacji. Analiza ta przyczyniła się do poprawy jakości i utrzymania standardów w projekcie.



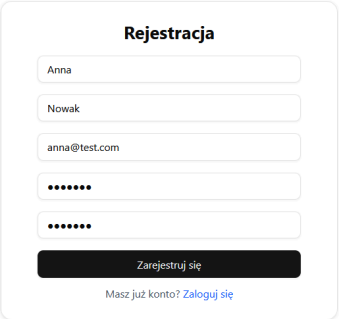
Przedstawienie działania

- Logowanie

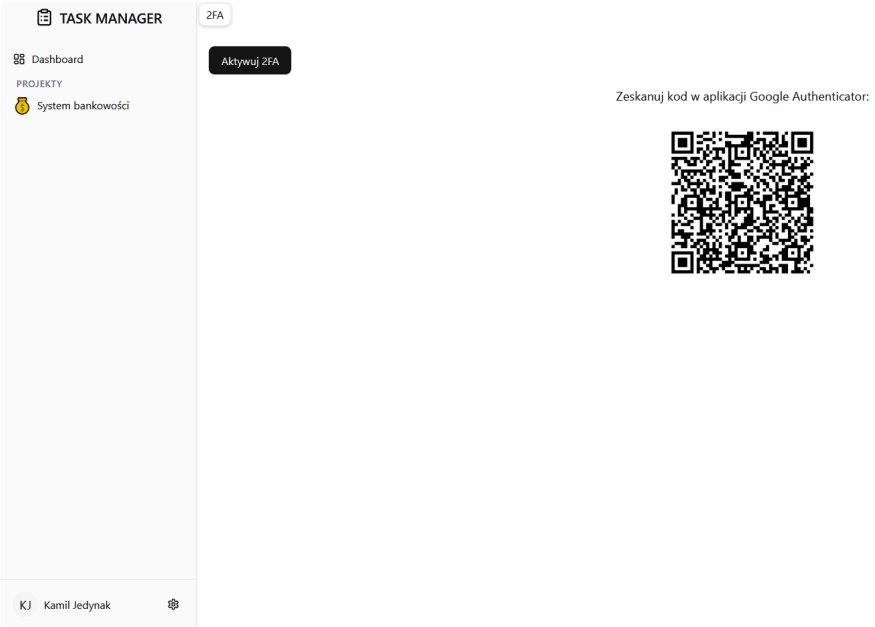


- Weryfikacja 2FA

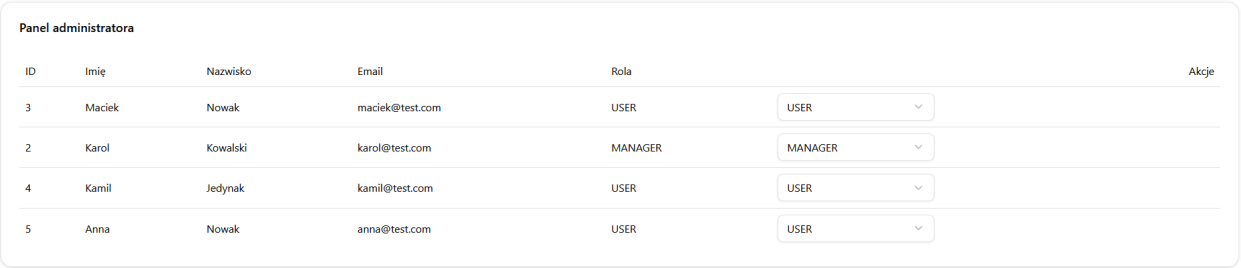
- Rejestracja



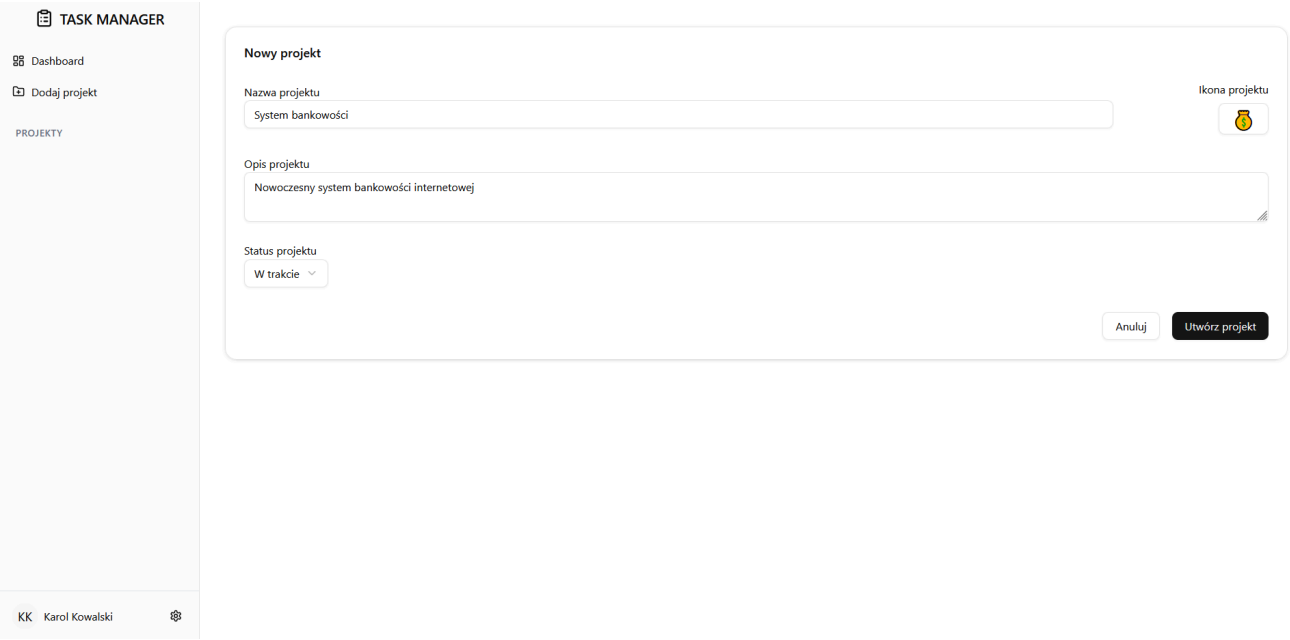
- Aktywacja 2FA



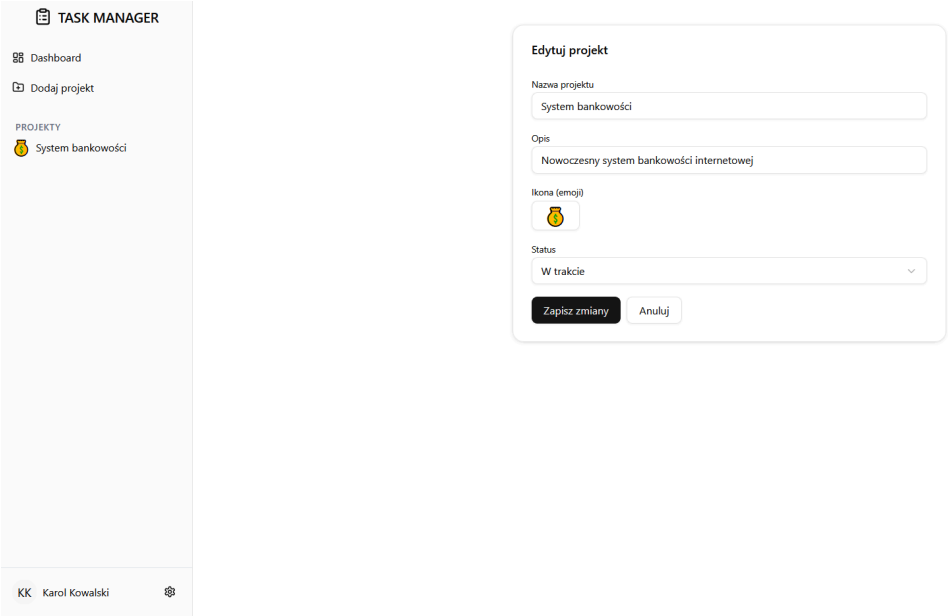
- Admin panel



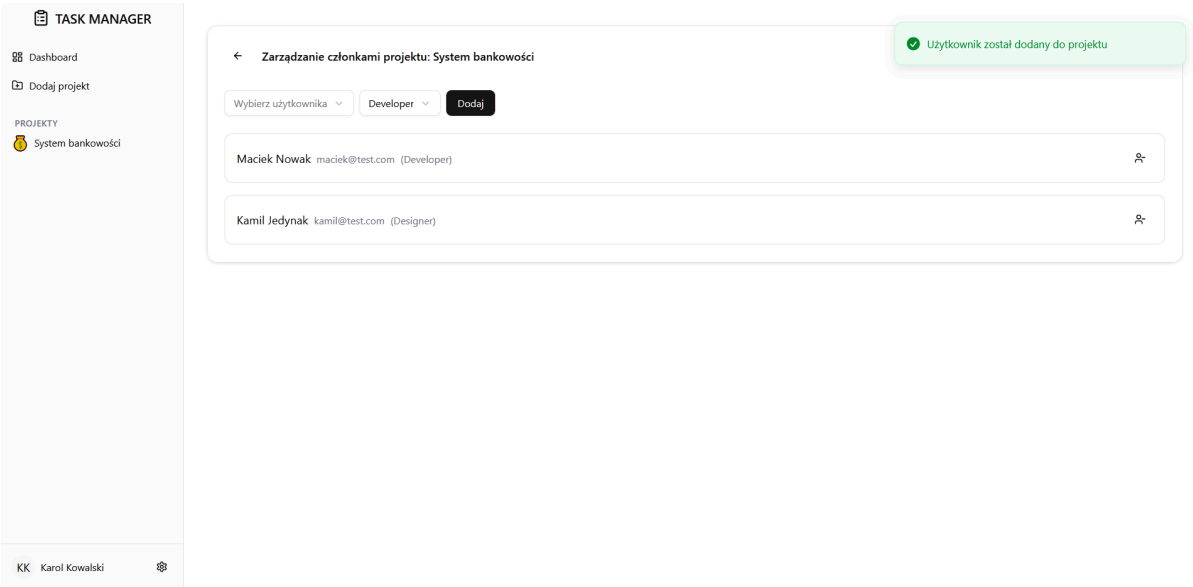
- Tworzenie projektu



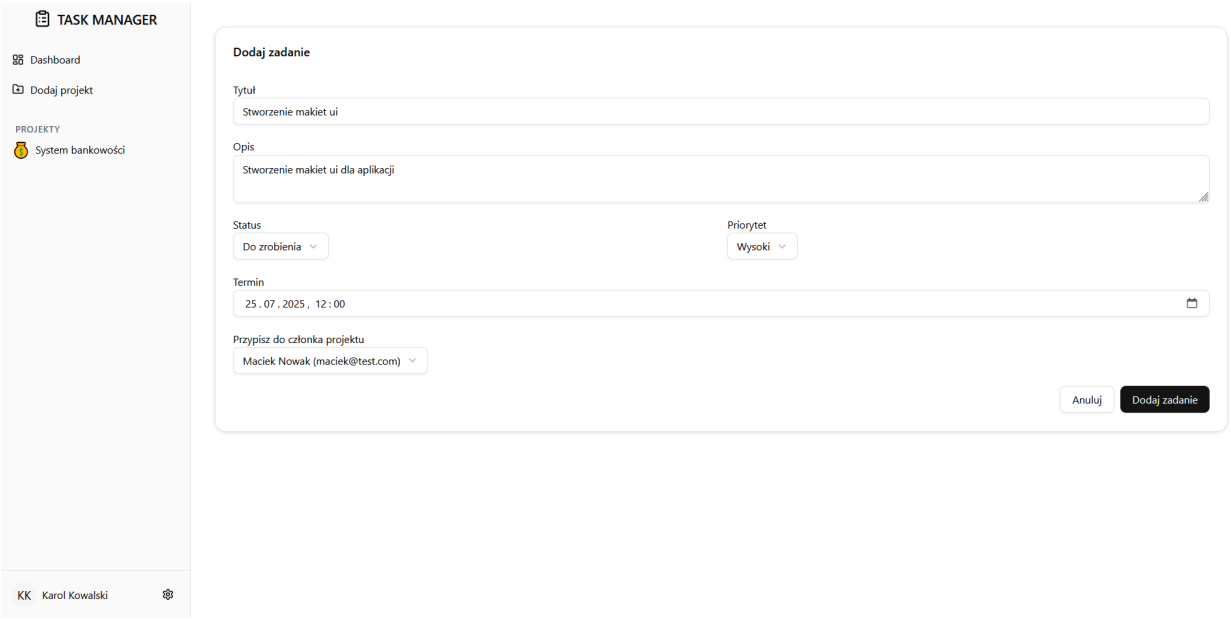
- Edytowanie projektu



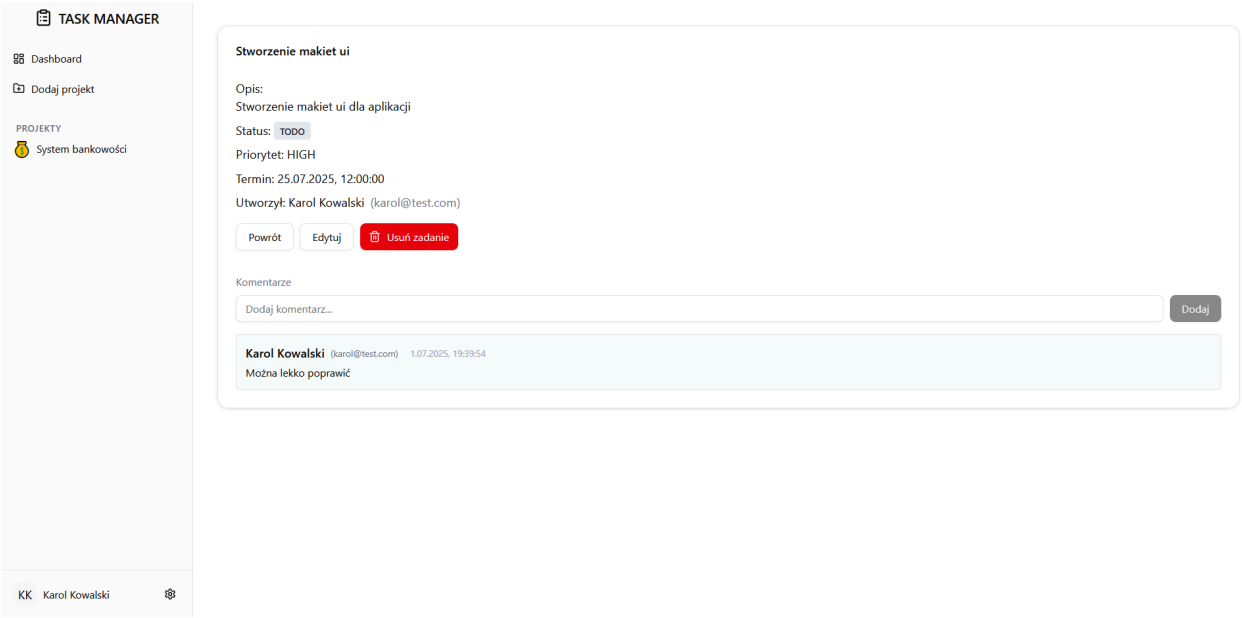
- Dodawanie użytkownika



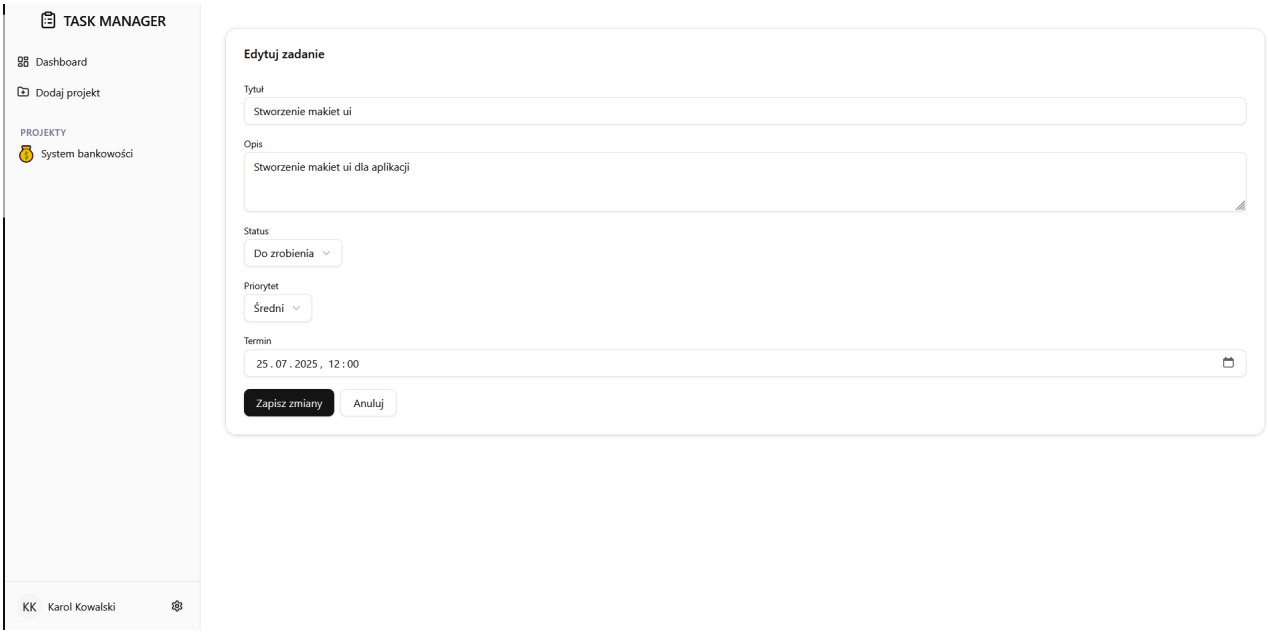
- Dodawanie zadania



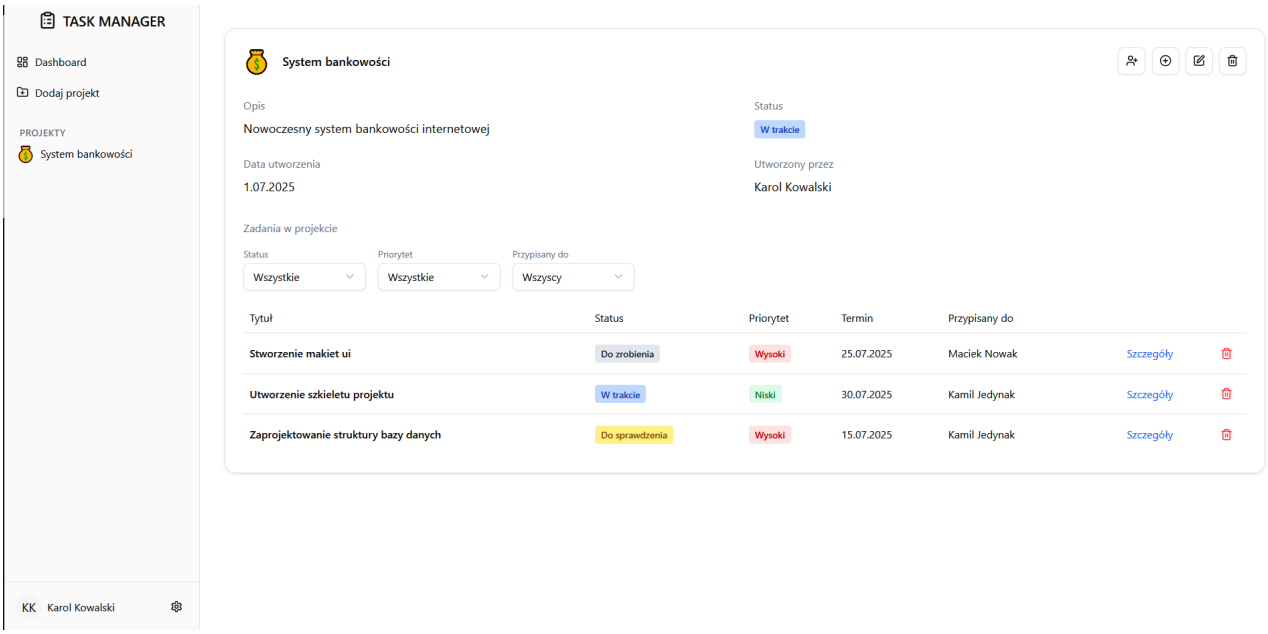
- Informacje o zadaniu



- Edytowanie zadania



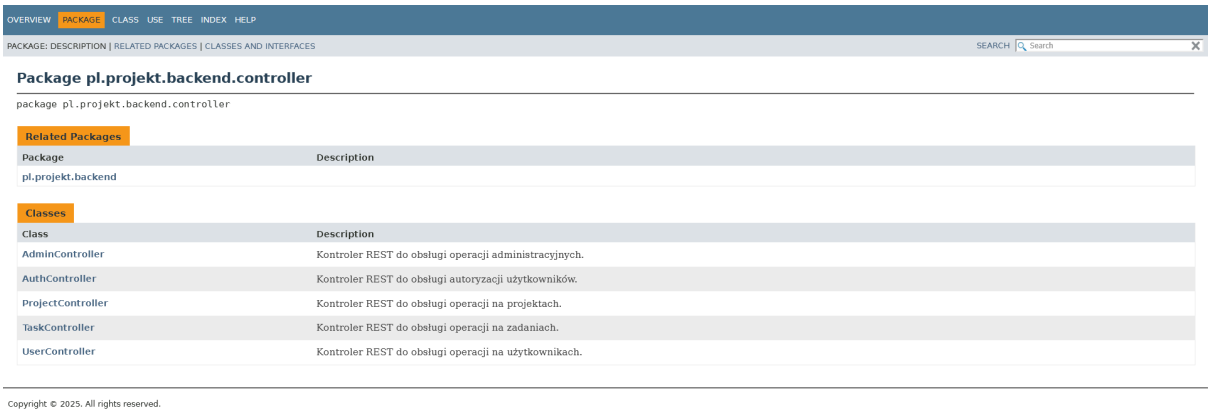
- Informacje o projekcie



Dokumentacja JavaDoc

Dla projektu została wygenerowana dokumentacja w formacie Javadoc, obejmująca wszystkie klasy i pakiety znajdujące się w strukturze projektu. Dokumentacja zawiera opisy klas, metod oraz ich przeznaczenia, co ułatwia zrozumienie struktury i działania aplikacji. Wygenerowane pliki dokumentacji można znaleźć w katalogu *target/site/apidocs*.

Poniżej znajduje się przykładowa strona z dokumentacji wygenerowanej dla pakietu pl.projekt.backend.controller, przedstawiająca zestaw kontrolerów REST obsługujących zapytania do systemu:



OVERVIEW PACKAGE CLASS USE TREE INDEX HELP

PACKAGE: DESCRIPTION | RELATED PACKAGES | CLASSES AND INTERFACES SEARCH Search

Package pl.projekt.backend.controller

package pl.projekt.backend.controller

Related Packages

Package	Description
pl.projekt.backend	

Classes

Class	Description
AdminController	Kontroler REST do obsługi operacji administracyjnych.
AuthController	Kontroler REST do obsługi autoryzacji użytkowników.
ProjectController	Kontroler REST do obsługi operacji na projektach.
TaskController	Kontroler REST do obsługi operacji na zadaniach.
UserController	Kontroler REST do obsługi operacji na użytkownikach.

Copyright © 2025. All rights reserved.

Przykładowa strona z dokumentacji javadoc

Dokumentacja Swagger UI

Dla projektu została przygotowana również interaktywna dokumentacja API z wykorzystaniem narzędzia Swagger UI, która umożliwia przeglądanie dostępnych endpointów, sprawdzanie ich działania oraz testowanie zapytań bezpośrednio z poziomu przeglądarki. Dokumentacja obejmuje wszystkie główne obszary systemu, takie jak: zadania, projekty, użytkownicy, logowanie oraz funkcje administratora.

Zadania Endpointy zadań

DELETE	/api/tasks/{id}	Usuwanie zadania
GET	/api/tasks/{id}	Pobieranie pojedynczego zadania
GET	/api/tasks/{taskId}/comments	Pobieranie komentarzy do zadania
GET	/api/tasks/{id}/details	Pobieranie szczegółów zadania z informacją o twórcy
GET	/api/tasks/project/{projectId}	Pobieranie zadań dla projektu
GET	/api/tasks/project/{projectId}/all	Pobieranie wszystkich zadań projektu (dla MANAGERA)
GET	/api/tasks/my	Pobieranie zadań zalogowanego użytkownika
PATCH	/api/tasks/{id}/to-review	Przekazanie zadania do sprawdzenia (zmiana statusu na TO_REVIEW)
POST	/api/tasks	Tworzenie nowego zadania
POST	/api/tasks/{taskId}/comments	Dodawanie komentarza do zadania
PUT	/api/tasks/{id}	Aktualizacja zadania

Projekt Endpointy projektów

DELETE	/api/projects/{id}	Usuwanie projektu
DELETE	/api/projects/{projectId}/members/{userId}	Usuwanie użytkownika z projektu
GET	/api/projects/{id}	Pobieranie projektu po ID
GET	/api/projects	Pobieranie wszystkich projektów użytkownika
GET	/api/projects/{projectId}/members	Pobieranie członków projektu
GET	/api/projects/my-member-projects	Pobieranie projektów, w których jestem członkiem
POST	/api/projects	Tworzenie nowego projektu
POST	/api/projects/{projectId}/members	Dodawanie użytkownika do projektu
PUT	/api/projects/{id}	Aktualizacja projektu

Użytkownik Endpointy użytkowników

- GET `/api/users` Pobieranie listy wszystkich użytkowników
- GET `/api/users/me` Pobieranie podstawowych informacji o użytkowniku

Logowanie Endpointy logowania i rejestracji

- POST `/api/auth/register` Rejestracja
- POST `/api/auth/login` Logowanie
- POST `/api/auth/2fa/verify` Weryfikacja 2FA
- POST `/api/auth/2fa/enable` Włączanie 2FA

Administrator Endpointy administratora

- GET `/api/admin/users` Pobieranie wszystkich użytkowników
- POST `/api/admin/change-role` Zmiania roli użytkownika

Stworzone punkty końcowe przedstawione w postaci Swagger API

Wnioski

Projekt stanowi kompletny system wspomagający zespołową pracę nad projektami, integrujący nowoczesne rozwiązania z zakresu bezpieczeństwa, zarządzania zadaniami oraz intuicyjny interfejs użytkownika. Aplikacja została zrealizowana w architekturze klient-serwer, w której backend oparty na Spring Boot odpowiada za logikę biznesową, natomiast frontend zbudowany w React zapewnia dynamiczne i płynne doświadczenie użytkownika.

W ramach systemu zaimplementowano bezpieczny mechanizm rejestracji i logowania z wykorzystaniem tokenów JWT, umożliwiający autoryzację użytkowników. Dodatkowo, w celu zwiększenia poziomu ochrony kont, wprowadzono możliwość skorzystania z dwuetapowego uwierzytelniania (2FA). Istotnym elementem systemu jest mechanizm ról użytkowników, który pozwala na precyzyjne zarządzanie dostępem do zasobów i operacji w aplikacji. Użytkownicy mogą być przypisani do ról takich jak np. zwykły użytkownik, manager lub administrator, co wpływa na ich uprawnienia w systemie.

Zaimplementowano pełen zestaw operacji CRUD (tworzenie, odczyt, edycja, usuwanie) dla projektów. W ramach zarządzania zadaniami użytkownicy mogą tworzyć, edytować i usuwać zadania w obrębie projektów, a także przypisywać je do konkretnych osób, ustalać terminy wykonania, priorytety oraz statusy.

W systemie przewidziano również panel administratora, który umożliwia modyfikację ich ról i uprawnień. Każdy wrażliwy endpoint został zabezpieczony przed nieautoryzowanym dostępem zgodnie z przyjętymi regułami autoryzacji. Wszystkie powyższe funkcjonalności zostały w pełni zintegrowane z aplikacją frontendową, dzięki czemu użytkownik może zarządzać projektami i zadaniami w czasie rzeczywistym, bez konieczności odświeżania strony.

W celu zapewnienia jakości, aplikacja została pokryta testami jednostkowymi z wykorzystaniem Spring Boot, co ułatwia dalszy rozwój i utrzymanie stabilności systemu. Elementem, którego nie udało się zrealizować, był system powiadomień – jego wdrożenie zostało zaplanowane, lecz z powodu ograniczeń czasowych nie zostało ukończone.